Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI
Email: vtu29748@veltech.edu.in
Roll no: vtu29748
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 5_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

Input Format

The first line of input consists of an integer n, denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

Output Format

vtu29748

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
4 1 5 3 2 6

Output: Given Array
4 1 5 3 2 6
After iteration 1
1 4 5 3 2 6
After iteration 2
1 4 5 3 2 6
After iteration 3
1 4 5 2 3 6
After iteration 4

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

Answer

```
#include <stdio.h>

int iteration=0;

void printArray(int arr[],int n){
   for(int i=0;i<n;i++) printf("%d ",arr[i]);
   printf("\n");
}

void merge(int arr[],int l,int m,int r,int n){
   int n1=m-l+1,n2=r-m;
   int L[n1],R[n2];
   for(int i=0;i<n1;i++) L[i]=arr[l+i];
   for(int j=0;j<n2;j++) R[j]=arr[m+1+j];
   int i=0,j=0,k=l;
   while(i<n1&&j<n2){
      if(L[i]<=R[j]) arr[k++]=L[i++];
      else arr[k++]=R[j++];
   }
   while(i<n1) arr[k++]=L[i++];
   while(j<n2) arr[k++]=R[j++];
   iteration++;
   printf("After iteration %d\n",iteration);
   printArray(arr,n);
}

void mergeSort(int arr[],int l,int r,int n){
   if(l<r){
      int m=(l+r)/2;
      mergeSort(arr,l,m,n);
      mergeSort(arr,m+1,r,n);
      merge(arr,l,m,r,n);
   }
}
```

```
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++) scanf("%d",&arr[i]);
    printf("Given Array\n");
    printArray(arr,n);
    mergeSort(arr,0,n-1,n);
    printf("Sorted Array\n");
    printArray(arr,n);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer n, representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)
- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

Output Format

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:
- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.
- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Harish

vtu29748

7.8
23
ece
Trell
8
21
cse
Kamalesh
7
25
it
Maaran
5.6
20
civil
age

Output: Sorted Student Records:

Name GPA Age Major

Maaran 5.60 20 civil

Trell 8.00 21 cse

Harish 7.80 23 ece

Kamalesh 7.00 25 it

Answer

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char name[50];
    float gpa;
    int age;
    char major[50];
} Student;

int criterion;

int compare(Student a, Student b) {
    if (criterion == 1) {
        if (a.gpa < b.gpa) return -1;
        else if (a.gpa > b.gpa) return 1;
        else return 0;
    } else if (criterion == 2) {
```

```
        return a.age - b.age;
    } else {
        return strcmp(a.major, b.major);
    }
}

void swap(Student *a, Student *b) {
    Student temp = *a;
    *a = *b;
    *b = temp;
}

int partition(Student arr[], int low, int high) {
    Student pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (compare(arr[j], pivot) <= 0) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(Student arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    Student arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i].name);
        scanf("%f", &arr[i].gpa);
        scanf("%d", &arr[i].age);
        scanf("%s", arr[i].major);
```

```
    }
    char crit[10];
    scanf("%s", crit);
    if (strcmp(crit, "gpa") == 0) criterion = 1;
    else if (strcmp(crit, "age") == 0) criterion = 2;
    else criterion = 3;

    quickSort(arr, 0, n - 1);

    printf("Sorted Student Records:\n");
    printf("Name\t\tGPA\t\tAge\t\tMajor\n");
    for (int i = 0; i < n; i++) {
        printf("%-10s\t%.2f\t%-3d\t\t%-15s\n",
               arr[i].name, arr[i].gpa, arr[i].age, arr[i].major);
    }
    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10
banana pineapple mango chikoo jack star guava lemon tomato orange
Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
#include <stdio.h>
#include <string.h>

void swap(char a[50], char b[50]) {
    char temp[50];
    strcpy(temp, a);
    strcpy(a, b);
    strcpy(b, temp);
}

void heapify(char arr[][50], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;

    if (right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(char arr[][50], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
```

```
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    char arr[20][50];
    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    heapSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%s ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8

8

Output:

7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

Input Format

The first line of input consists of an integer n, representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x, representing the specific book ID to find.

Output Format

The output displays indexes of the first and last occurrence of x, as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
1 2 2 2 2 3 4 8 8 8
8

Output: 7 9

Answer

```
#include <stdio.h>

int firstOccurrence(int arr[], int l, int r, int x) {
    if (l > r) return -1;
    int mid = (l + r) / 2;
```

```
    if ((mid == 0 || arr[mid - 1] < x) && arr[mid] == x)
        return mid;
    else if (arr[mid] >= x)
        return firstOccurrence(arr, l, mid - 1, x);
    else
        return firstOccurrence(arr, mid + 1, r, x);
}

int lastOccurrence(int arr[], int l, int r, int x, int n) {
    if (l > r) return -1;
    int mid = (l + r) / 2;
    if ((mid == n - 1 || arr[mid + 1] > x) && arr[mid] == x)
        return mid;
    else if (arr[mid] > x)
        return lastOccurrence(arr, l, mid - 1, x, n);
    else
        return lastOccurrence(arr, mid + 1, r, x, n);
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    int x;
    scanf("%d", &x);

    int first = firstOccurrence(arr, 0, n - 1, x);
    int last = lastOccurrence(arr, 0, n - 1, x, n);

    if (first == -1 || last == -1)
        printf("NO OCCURRENCES\n");
    else
        printf("%d %d\n", first, last);

    return 0;
}
```

Status : Correct **Marks : 10/10**